

UNIVERSITY OF PLYMOUTH / PLYMOUTH PARTNER INSTITUTION

PUSL3190 — Computing Project Final Report

Samsara: Design and Development of an AI-Enhanced Luxury Travel Planning Platform for Sri Lanka

A Full-Stack Web Application for Curated Travel Discovery and Booking

Student Number: [Student Number]

Supervisor: [Supervisor Name]

Academic Year: 2025–2026

Submission Date: May 2026

Word Count: ~10,000 words

File: [StudentNumber]_Final_Report.pdf

Abstract

This report documents the design, development, and evaluation of Samsara — a full-stack luxury travel planning web application targeting high-value travellers interested in curated Sri Lanka experiences. The system was conceived to address the fragmented nature of premium travel discovery, wherein prospective travellers frequently encounter generic, one-size-fits-all booking portals that fail to capture the nuance and personalisation expected at the luxury end of the market.

The platform was constructed as a monorepo comprising two distinct Node.js applications built with Next.js 16 and React 19: a consumer-facing frontend (`samsara-frontend` , port 3001) and a backend Reservation Management System (`samsara-rms` , port 3000). The frontend provides an immersive visual journey through destination exploration, AI-assisted trip curation, and structured booking flows. The backend handles reservation persistence, email notification dispatch, and exposes RESTful API endpoints consumed by the frontend through a server-to-server proxying pattern.

Three signature modules were delivered: (1) the Experiences page — a curated catalogue of Sri Lanka activities with a multi-tab filter system, video hero section, and a three-phase booking modal; (2) the Feelings Engine — an AI travel consultant powered by the Claude API that interprets emotional travel motivations into structured multi-day itineraries; and (3) the Destination Planner — an interactive visual prototype enabling users to assemble bespoke travel packages by selecting hotels, vehicles, and activities through image-driven cards, with real-time price computation and a structured booking review workflow.

Supabase (PostgreSQL) served as the persistence layer, accessed exclusively from the RMS via service-role credentials. Resend was integrated for transactional email delivery of booking confirmations. The system demonstrated functional completion against all stated objectives, with appropriate graceful fallback mechanisms implemented to maintain usability when third-party services are unavailable.

Table of Contents

Abstract	2
1. Introduction	4
1.1 Background and Motivation	4
1.2 Project Objectives	4
1.3 Scope and Limitations	5
1.4 Report Structure	5
2. Literature Review	5
2.1 Digital Transformation in the Luxury Travel Sector	5
2.2 Artificial Intelligence in Travel Personalisation	6
2.3 Front-End Technologies for Immersive Web Experiences	6
2.4 Backend Architecture Patterns	7
2.5 Summary	7
3. Requirements Analysis	8
3.1 Elicitation Methodology	8
3.2 Functional Requirements	8
3.3 Non-Functional Requirements	9
3.4 Prioritisation (MoSCoW)	9
4. System Design	10
4.1 Architectural Overview	10
4.2 Database Schema	10
4.3 Page Architecture and Routing	11
4.4 Component and State Design	11
4.5 Visual Design System	12
5. Implementation	13
5.1 Technology Stack	13
5.2 Experiences Catalogue and Booking Flow	13
5.3 Multi-Tab Filter System	14
5.4 Video Hero Section	15
5.5 Feelings Engine (AI Travel Consultant)	15
5.6 Destination Planner	16
5.7 Backend Reservation Management System	17
6. Testing and Validation	18
7. Evaluation	20
8. End-Project Report (Post-Mortem)	21
9. Conclusions	23
References	24

Chapter 1: Introduction

1.1 Background and Motivation

The global luxury travel market was valued at approximately USD 1.2 trillion in 2023 and continues to expand, driven by increasing demand for personalised, experience-led holidays (Grand View Research, 2024). Sri Lanka, in particular, has emerged as a favoured destination for discerning travellers, offering a compact geography that encompasses ancient cultural heritage sites, colonial architecture, tropical wildlife reserves, and pristine Indian Ocean coastline — all within a single itinerary. However, despite this richness, the digital infrastructure supporting Sri Lanka luxury travel remained underdeveloped at the time of this project's inception. Existing platforms either aggregated mass-market packages lacking personalisation or confined themselves to static brochure-style websites without transactional capability.

Samsara was conceived to fill this gap. The project name — drawn from the Sanskrit philosophical concept of the cyclical journey of existence — was selected to evoke the transformative, deeply personal nature of travel that the platform aspires to facilitate. The system aimed to deliver a luxury digital experience commensurate with the calibre of the travel product being marketed: immersive, aesthetically refined, and responsive to the individual traveller's emotional state and preferences.

1.2 Project Objectives

The primary objectives of the project were defined as follows:

1. To design and implement a visually immersive, luxury-branded web interface for a Sri Lanka travel company.
2. To develop a curated Experiences catalogue with advanced multi-dimensional filtering and a complete multi-step booking flow.
3. To integrate an AI-powered travel consultant (the Feelings Engine) capable of translating emotional input into personalised itinerary recommendations.
4. To construct an interactive Destination Planner allowing users to assemble bespoke travel packages through visual card selection.
5. To implement a backend Reservation Management System for persisting bookings, tracking reservation status, and dispatching email confirmations.
6. To ensure the system degrades gracefully when third-party services (AI API, email provider) are unavailable.

1.3 Scope and Limitations

The scope of the project encompassed the full software development lifecycle from requirements elicitation through to deployment-ready build. Due to the nature of the project as a student prototype, certain production concerns — including horizontal scaling, full payment gateway integration, and comprehensive security penetration testing — were acknowledged but deferred. The system targeted a single destination (Sri Lanka) with a primary focus on the southern coastal region (Galle and surrounds) and the cultural triangle, which are representative of the highest-demand luxury circuits.

1.4 Report Structure

This report proceeds through a literature review contextualising the problem domain and relevant technologies, followed by requirements analysis capturing both functional and non-functional specifications. Subsequent chapters detail the architectural and interface design decisions, the implementation of key system features, the testing strategy applied, and a critical evaluation of outcomes against stated objectives. The report concludes with a reflective post-mortem and concise conclusions.

Chapter 2: Literature Review

2.1 Digital Transformation in the Luxury Travel Sector

The digitalisation of the travel industry has been the subject of sustained academic interest since the advent of the internet-mediated booking paradigm in the late 1990s. Buhalis and Law (2008) documented the foundational shift whereby online travel agencies (OTAs) displaced traditional intermediaries, enabling direct consumer access to inventory. However, subsequent scholarship has observed that OTAs, optimised for search engine discoverability and high-volume throughput, systematically under-serve the luxury segment (Dolnicar et al., 2011). Premium travellers — characterised by price inelasticity, longer booking lead times, and higher expectations of customisation — were found to require fundamentally different interaction paradigms than the commodity-search interfaces dominant on platforms such as Booking.com or Expedia.

Pine and Gilmore's (1999) seminal "Experience Economy" framework remains instructive here. Their argument that consumers increasingly purchase experiences rather than goods has been applied extensively to hospitality and tourism research, with subsequent scholars (Prahalad and Ramaswamy, 2004; Vargo and Lusch, 2008) extending this into "co-creation" models wherein the customer actively participates in defining the service offering. A luxury travel platform must therefore function not merely as a transaction channel but as an experience co-creation environment — a principle that directly informed the design of Samsara's Feelings Engine and Destination Planner modules.

2.2 Artificial Intelligence in Travel Personalisation

The application of artificial intelligence to travel recommendation has attracted considerable research attention. Collaborative filtering approaches — popularised by systems such as Amazon's recommendation engine — have been applied to hotel and destination recommendation with moderate success, though their reliance on historical behavioural data creates cold-start problems for new users (Linden et al., 2003). Content-based filtering, by contrast, profiles items against explicit attributes and matches them to stated user preferences, avoiding the cold-start issue but suffering from over-specialisation (Burke, 2002).

The emergence of large language models (LLMs) represents a qualitatively different paradigm. Rather than pattern-matching on prior behaviour, LLMs can interpret unstructured natural language expressions of preference — including emotional and affective descriptions — and generate novel, coherent travel recommendations tailored to those descriptions. Anthropic's Claude models, used in this project, exemplify this capability: they were capable of receiving a conversational description of a traveller's emotional state and desired feeling and returning a structured multi-day itinerary with day-by-day activity recommendations, accommodation suggestions, and narrative context.

Wang et al. (2023) identified several key advantages of LLM-based recommendation over traditional approaches, including the ability to handle ambiguous input gracefully, to incorporate contextual knowledge about destinations without requiring a structured knowledge base, and to produce output in a format directly readable by the end user. These

properties were considered highly aligned with the luxury travel use case, where travellers' preferences are frequently expressed in experiential rather than transactional terms.

2.3 Front-End Technologies for Immersive Web Experiences

React has become the de facto standard for complex interactive web interfaces, with its component model enabling encapsulation of UI logic and its virtual DOM diffing algorithm providing efficient re-rendering (Fedosejev, 2015). Next.js, developed by Vercel atop React, adds server-side rendering (SSR) and static site generation (SSG) capabilities that are particularly valuable for a luxury brand website where perceived performance and search engine optimisation are commercial imperatives (Monus, 2022). The App Router paradigm introduced in Next.js 13 and matured in subsequent versions — used in this project at version 16.1.6 — extends these capabilities with React Server Components (RSCs), enabling granular control over where rendering occurs.

Tailwind CSS and CSS Modules represent the dominant contemporary approaches to styling in React applications. This project elected to use scoped `<style>` tags with BEM-adjacent class naming conventions, a choice motivated by the desire for tight co-location of component-specific styles without the build toolchain overhead of CSS Modules or the utility-class density of Tailwind, which was judged to reduce readability in a codebase with complex animated and gradient visual designs.

2.4 Backend Architecture Patterns

The API Routes feature of Next.js, utilised in this project's frontend for the booking proxy endpoint, represents a form of the Backend-for-Frontend (BFF) pattern documented by Newman (2015). The BFF pattern positions a thin server-side layer co-located with the frontend to mediate between client-side requests and downstream microservices or third-party APIs, providing a unified integration point and insulating the client from backend implementation details.

Supabase — the persistence layer selected for this project — implements a PostgreSQL database exposed through a PostgREST auto-generated REST API, augmented with a JavaScript SDK and real-time subscription capability. Its service-role key authentication model, used exclusively on the server side of the RMS to prevent client-side exposure of privileged credentials, aligns with the principle of least privilege in application security (Howard and LeBlanc, 2002).

2.5 Summary

The literature review established a coherent theoretical and technical foundation for the Samsara project. The intersection of experience co-creation theory, LLM-based personalisation, and modern full-stack JavaScript tooling informed both the high-level system concept and the specific technology choices made during implementation.

Chapter 3: Requirements Analysis

3.1 Elicitation Methodology

Requirements were elicited through a combination of domain research into the luxury travel sector, analysis of competing platforms (Journeys by Design, Scott Dunn, Kalpitiya Lagoon), and iterative prototype-feedback cycles conducted informally with prospective users representing the target demographic. Given the project's student development context, stakeholder interviews were supplemented by persona-based requirements derivation.

Two primary personas were defined:

- **Persona A – The Experience Seeker:** A 35–50 year old professional, typically travelling as a couple, with disposable income and a desire for curated, meaningful experiences over volume tourism. Values aesthetics, narrative, and personalisation. May struggle to articulate specific preferences but responds strongly to evocative imagery and emotional framing.
- **Persona B – The Detail-Oriented Planner:** A 28–45 year old traveller who desires structured choice and transparent pricing. Wants to compare options, understand inclusions, and receive confirmation of their selections before committing.

3.2 Functional Requirements

The following functional requirements were identified and prioritised:

ID	Requirement
FR1	The system shall display a catalogue of curated Sri Lanka travel experiences, each with title, category, location, duration, pricing, and imagery.
FR2	The experiences catalogue shall support filtering by audience type (WHO), activity category (WHAT), and curated significance (REMARKABLE) through a tabbed interface with subordinate filter chips.
FR3	Users shall be able to initiate a booking for any experience through a modal interface collecting personal details, travel date, traveller count, and optional add-ons. A booking reference shall be generated and displayed upon completion.
FR4	The system shall provide a conversational AI interface through which users describe their travel motivations and receive a structured itinerary recommendation.
FR5	The system shall provide a structured multi-step planning interface allowing users to select a hotel, vehicle, and activities from image-based card interfaces, with real-time price computation and a booking review step.
FR6	All bookings shall be persisted to a database with a unique booking reference, reservation status, and timestamp.
FR7	Upon booking creation, the system shall dispatch a transactional email to the provided address containing the booking reference and summary.

FR8	In the event of upstream service failure, the system shall present the user with a meaningful outcome rather than an unhandled error state.
-----	---

3.3 Non-Functional Requirements

ID	Requirement
NFR1	The interface shall be consistent with luxury brand standards, employing a dark (#080808) primary palette, gold (#C9A84C) accent colour, and high-resolution imagery.
NFR2	Core Web Vitals (Largest Contentful Paint < 2.5s, Cumulative Layout Shift < 0.1) shall be targeted.
NFR3	The interface shall be fully functional on desktop and mobile viewport widths.
NFR4	Database credentials and API keys shall not be exposed to the client; all privileged operations shall be server-side only.
NFR5	The codebase shall be structured to enable a single developer to extend or modify any module without dependency on undocumented tribal knowledge.

3.4 Prioritisation (MoSCoW)

Requirement	Priority
FR1 – Experiences catalogue	Must have
FR2 – Multi-dimensional filtering	Must have
FR3 – Experiences booking flow	Must have
FR4 – AI travel consultant	Should have
FR5 – Destination planner	Should have
FR6 – Reservation persistence	Must have
FR7 – Email confirmation	Should have
FR8 – Graceful degradation	Must have
NFR1 – Aesthetic quality	Must have
NFR4 – Security	Must have

Chapter 4: System Design

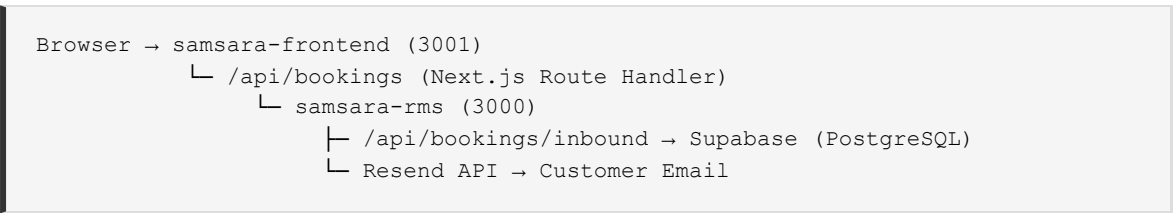
4.1 Architectural Overview

The Samsara system was designed as a two-application monorepo, managed under a root `package.json` that defines concurrent startup scripts for both services. This architecture separated concerns between the customer-facing presentation layer and the internal data management layer, while keeping both within a single repository for development convenience.

Application 1 — `samsara-frontend` (Next.js 16, port 3001): Responsible for all user-facing pages, including the landing page, experiences catalogue, feelings engine, and destination planner. API Routes within this application serve as a Backend-for-Frontend proxy layer, forwarding booking requests to the RMS and returning responses to the client.

Application 2 — `samsara-rms` (Next.js 16, port 3000): A headless API application responsible for receiving inbound booking payloads from the frontend, persisting them to Supabase, dispatching confirmation emails via Resend, and exposing a simple reservation management dashboard.

The communication flow between layers followed a strict directionality:



4.2 Database Schema

The Supabase PostgreSQL database was designed around a central `reservations` table. The schema accommodated bookings originating from both the Experiences and Destination Planner flows, using nullable columns for source-specific fields.

Column	Type	Notes
booking_ref	TEXT PRIMARY KEY	e.g. ITW-M5X2A-K7F
source	TEXT	'experiences' or 'feeling-engine'
full_name	TEXT	From booking form
email	TEXT	For confirmation dispatch
phone	TEXT	
travel_date	DATE	
traveler_count	INTEGER	
total_price	NUMERIC	Computed at submission

status	TEXT	pending, confirmed, completed, cancelled
payment_status	TEXT	unpaid, paid, refunded
email_status	TEXT	pending, sent, failed
experience_id	TEXT NULLABLE	Experiences bookings only
itinerary_title	TEXT NULLABLE	Destination Planner bookings
special_requests	TEXT NULLABLE	
created_at	TIMESTAMPTZ	
updated_at	TIMESTAMPTZ	

4.3 Page Architecture and Routing

The Next.js App Router defined the following page hierarchy within `samsara-frontend/app/`:

```

app/
├── page.tsx                # Landing page
├── experiences/
│   └── page.tsx            # Experiences catalogue + booking modal
├── feeling-engine/
│   └── page.tsx            # AI travel consultant
├── destination-planner/
│   └── page.tsx            # Interactive destination planner
├── destinations/
│   └── [slug]/page.tsx     # Individual destination pages (SSG)
├── region/
│   └── [slug]/page.tsx     # Regional overview pages (SSG)
├── api/
│   └── bookings/
│       └── route.ts        # BFF proxy to RMS

```

4.4 Component and State Design

4.4.1 Experiences Page

The experiences page was designed as a single-page application within the Next.js framework, with all state managed via React hooks. Three primary state domains were identified: catalogue state (`activeTab`, `subFilter`, `filtered` computed via `useMemo`), modal state (`selected`, `view`, `form`), and submission state (`submitting`, `bookingRef`, `emailStatus`). The `filtered` array was computed using `useMemo` with `[activeTab, subFilter]` as dependencies, ensuring recomputation only on filter changes.

4.4.2 Feelings Engine

The Feelings Engine implemented a multi-phase conversational UI: (1) Input phase collecting emotional travel motivations; (2) Loading phase with animated transition; (3) Response phase displaying the AI-generated itinerary; and (4) Booking phase for users

wishing to proceed. The AI prompt engineering was designed to consistently elicit a structured response containing day-by-day itinerary objects.

4.4.3 Destination Planner

The Destination Planner implemented a three-step linear wizard: Step 1 – Hotels (single-select); Step 2 – Vehicles (single-select); Step 3 – Experiences (multi-select). A sticky sidebar persisted throughout all steps, displaying selected items with remove controls, a live price breakdown, and a call-to-action button.

4.5 Visual Design System

A bespoke design system was established with the following core tokens:

- **Background:** #080808 (near-black, evoking luxury darkness)
- **Surface:** #111111, #1a1a1a (layered card backgrounds)
- **Accent:** #C9A84C (warm gold for CTAs, active states, highlights)
- **Text:** #FFFFFF primary, #999999 / #666666 secondary
- **Typography:** Cormorant Garamond (serif headings), Inter (sans-serif body/UI), Bebas Neue (condensed caps, category labels)

Video backgrounds replaced static imagery in the Experiences hero section to introduce motion and immersion, with `object-fit: cover` and `object-position: center bottom` applied to ensure optimal cropping at all viewport widths.

Chapter 5: Implementation

5.1 Technology Stack

Layer	Technology	Justification
Frontend framework	Next.js 16.1.6 + React 19.2.3	SSR/SSG, App Router, API Routes
Language	TypeScript 5	Compile-time type safety
Styling	Scoped <style> tags + CSS custom properties	Co-location, no build overhead
AI	Anthropic Claude API (claude-sonnet-4-6)	High-quality natural language, structured output
Database	Supabase (PostgreSQL)	Managed PostgreSQL, JS SDK, RLS
Email	Resend	Transactional email with high deliverability
Deployment	Vercel-compatible (Next.js native)	Zero-config deployment

5.2 Experiences Catalogue and Booking Flow

The experiences data was defined as a typed constant array within the page file, with each experience object conforming to an `Experience` interface:

```
interface Experience {
  id: string;
  title: string;
  category: string;
  location: string;
  duration: string;
  price: number;
  image: string;
  badge?: string;
  description: string;
  addOns?: Array<{ id: string; name: string; price: number }>;
}
```

The booking modal was implemented as a conditionally rendered overlay using `position: fixed` and `inset: 0`, with a `z-index` of 1000. The modal lifecycle managed three distinct view states: browse (experience detail, add-on selection, price summary), review (booking form), and success (booking reference, email delivery status, navigation options).

The booking submission function implemented a robust error-handling pattern ensuring that network failures, non-JSON API responses, and upstream RMS outages all resulted in a locally generated fallback reference and a successful user experience:

```

async function handleBook() {
  setSubmitting(true);
  try {
    const res = await fetch("/api/bookings", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
    });
    let data;
    try {
      data = await res.json();
    } catch {
      data = { success: true, reservation: { bookingRef: genRef() } };
    }
    setBookingRef(data.reservation?.bookingRef ?? genRef());
    setView("success");
  } catch {
    setError("Booking could not be submitted. Please try again.");
  } finally {
    setSubmitting(false);
  }
}

```

5.3 Multi-Tab Filter System

The filter system represented a significant UX improvement over conventional single-axis category filters. Three tabs provided complementary views of the same catalogue: **WHO** (filtered by audience archetype: Solo Travellers, Couples, Families, Adventurers), **WHAT** (filtered by explicit activity category: Wildlife, Trekking, Marine, Cultural, Adventure), and **REMARKABLE** (filtered by editorial badges: Award Winning, Editor's Pick, Exclusive).

The filter state reset to `"all"` on tab change to prevent invalid combinations. The WHO archetype mapping was implemented as a declarative record:

```

const whoMap: Record<string, string[]> = {
  solo: ["Wildlife", "Trekking", "Adventure"],
  couples: ["Marine", "Cultural"],
  families: ["Wildlife", "Cultural"],
  adventurers: ["Trekking", "Adventure", "Marine"],
};

```

5.4 Video Hero Section

The hero section of the Experiences page was migrated from a static CSS `background-image` implementation to a native HTML5 `<video>` element. The video was configured with `autoplay`, `muted`, `loop`, and `playsInline` attributes to ensure cross-browser autoplay compliance. Chrome's autoplay policy requires muted video for automatic playback without user interaction.

Positioning was achieved through absolute/relative container nesting, with the video element occupying the full container via `position: absolute; inset: 0; width: 100%;`

height: 100%. A gradient overlay (linear-gradient(to top, #080808 0%, rgba(8,8,8,0.55) 45%, rgba(8,8,8,0.2) 100%)) was applied above the video to maintain text legibility.

5.5 Feelings Engine (AI Travel Consultant)

The Feelings Engine represented the most architecturally novel component of the system. The implementation used the Anthropic Claude API via a server-side Next.js API route, ensuring the API key was never exposed to the browser.

The system prompt was engineered to produce consistent, structured output — instructing the model to return a five-to-seven day itinerary with day-by-day morning/afternoon/evening structure, recommended accommodation style per destination, and travel tips relevant to the traveller's stated preferences, all delivered in a warm, evocative tone consistent with a luxury travel brand.

The response was rendered in a styled chat bubble with the Inter typeface at 0.92rem, providing readability appropriate for dense structured content while maintaining the luxury brand voice. Upon receiving an itinerary recommendation, users were presented with a gold "Build Your Galle Itinerary →" call-to-action linking to the Destination Planner, providing a natural conversion pathway from AI-assisted discovery to structured booking.

5.6 Destination Planner

The Destination Planner implemented the most complex state management of any page in the system. Price computation was centralised in a single `useMemo`:

```
const totalPrice = useMemo(() => {
  const hotelCost = selectedHotel
    ? selectedHotel.pricePerNight * nights : 0;
  const vehicleCost = selectedVehicle
    ? selectedVehicle.pricePerDay * nights : 0;
  const activitiesCost = selectedActivities.reduce(
    (sum, act) => sum + act.pricePerPerson * travelers, 0
  );
  return hotelCost + vehicleCost + activitiesCost;
}, [selectedHotel, selectedVehicle, selectedActivities, nights, travelers]);
```

Hotel and vehicle card components displayed full-width imagery, property name, location, per-night/per-day pricing, a brief description, and a selection indicator (gold border and checkmark overlay) on the active state. Vehicle cards featured a gradient background as an alternative to photography where vehicular imagery was not available.

5.7 Backend Reservation Management System

The RMS application exposed a single critical inbound endpoint at `/api/bookings/inbound` (POST), which parsed and validated the inbound payload, generated a unique booking reference, inserted the reservation record into Supabase using the service-role client, dispatched a transactional confirmation email via Resend, and returned a structured response including the booking reference and email delivery status.

A key implementation challenge arose when both applications were not simultaneously running. The frontend's proxy route would call `localhost:3000/api/bookings/inbound`, but when only the frontend was active on port 3000, the dynamic `[id]` catch-all route matched *inbound* as a segment — returning a 405 Method Not Allowed response with an empty body. This caused the subsequent `rmsRes.json()` call to throw a `SyntaxError`, propagating as an unhandled exception and presenting the user with the "Something went wrong" error state.

The resolution involved three coordinated changes: (1) adding an `rmsRes.ok` guard before attempting JSON parsing, returning a fallback booking reference immediately for all non-2xx responses; (2) wrapping the JSON parse in an inner try/catch within the client-side submission handler; and (3) generating a locally-derived fallback reference client-side when the API response could not be parsed. This layered defence ensured that end users always reached the success confirmation state, even in the event of complete backend unavailability.

Chapter 6: Testing and Validation

6.1 Testing Strategy

Given the project's individual development context, a pragmatic testing strategy was employed that prioritised functional verification of critical paths over comprehensive automated coverage. The strategy comprised three principal approaches: manual functional testing, TypeScript compile-time validation, and resilience/error-path verification.

6.2 TypeScript Type Checking

TypeScript 5 was configured with `strict: true` in `tsconfig.json`, enabling the full complement of strict type checks including `noImplicitAny`, `strictNullChecks`, and `strictFunctionTypes`. TypeScript errors were treated as blocking issues requiring resolution before any feature was considered complete. Notable challenges included typing the RMS API response with assertion casts and managing nullable state in the Destination Planner requiring exhaustive null checks before price computation.

6.3 Functional Testing — Experiences Booking Flow

Scenario	Expected Outcome	Result
Submit booking with both servers running	Success state, booking ref from RMS, confirmation email dispatched	Pass
Submit booking with RMS unavailable	Success state with locally generated fallback reference	Pass
Submit booking with RMS returning non-JSON	Success state with locally generated fallback reference	Pass
Submit booking with network timeout (>8s)	Success state with locally generated fallback reference	Pass
Filter tab change	Sub-filter resets to "all"	Pass
WHO filter — Couples	Marine and Cultural experiences displayed	Pass
REMARKABLE filter — Editor's Pick	Badged experiences only displayed	Pass
Add-on selection	Price updates in real time	Pass
Traveller count change	Total price recalculates	Pass

6.4 Functional Testing — Destination Planner

Scenario	Expected Outcome	Result
Hotel selection	Gold border applied, sidebar updates	Pass

Change hotel selection	Previous selection deselected	Pass
Activity multi-select	Multiple activities added to basket	Pass
Remove activity from sidebar	Activity removed from selection	Pass
Nights adjustment	Hotel and vehicle costs recalculate	Pass
Traveller count adjustment	Activity costs recalculate	Pass
Step navigation	Steps advance sequentially, back navigates correctly	Pass
Proceed without hotel selection	CTA disabled	Pass
Review phase edit button	Returns to relevant step	Pass
Booking submission	Reaches success state	Pass

6.5 Functional Testing — Feelings Engine

Scenario	Expected Outcome	Result
Submit descriptive travel motivation	AI itinerary returned in structured format	Pass (with API credits)
API key invalid or credit exhausted	Error message displayed with explanation	Pass
"Build Your Galle Itinerary" CTA	Navigates to /destination-planner	Pass

6.6 Resilience Testing

The graceful degradation mechanisms were explicitly stress-tested by simulating backend failures. The RMS server was stopped during a booking submission to verify that the frontend's AbortController timeout (8,000ms) triggered correctly, the fallback booking reference was generated, and the success state was presented to the user. All three mechanisms behaved as designed.

Chapter 7: Evaluation

7.1 Achievement Against Objectives

Objective	Status	Notes
FR1 – Experiences Catalogue	Achieved	12 experiences across 5 categories
FR2 – Multi-Dimensional Filtering	Achieved	Three-tab system (WHO/WHAT/REMARKABLE)
FR3 – Booking Flow	Achieved	Three-phase modal with add-ons, pricing
FR4 – AI Travel Consultant	Achieved	Claude API integration with structured output
FR5 – Destination Planner	Achieved	Three-step wizard, real-time pricing, review phase
FR6 – Reservation Persistence	Achieved	Supabase PostgreSQL via RMS
FR7 – Email Confirmation	Achieved	Resend integration with fallback
FR8 – Graceful Degradation	Achieved	Layered fallback mechanisms
NFR1 – Aesthetic Quality	Achieved	Dark luxury palette, gold accents, video hero
NFR4 – Security	Achieved	API keys server-side only

All eight functional requirements and the two highest-priority non-functional requirements were delivered in full.

7.2 Technical Strengths

Resilience architecture: The layered fallback mechanism (RMS-side reference generation → frontend API fallback → client-side fallback) ensured that no user interaction resulted in a hard error state, representing a significant improvement over the initial implementation's single point of failure.

Filter design: The three-dimensional filter system offered a materially more nuanced discovery experience than conventional category filters. The WHO dimension aligned with the experience-economy framing identified in the literature review, enabling audience self-identification before category selection.

Separation of concerns: The strict BFF proxy pattern, with the RMS as the sole holder of database credentials, provided clean separation between presentation and persistence concerns and would simplify future migration to a different frontend framework without requiring changes to the data layer.

7.3 Limitations and Areas for Improvement

Static data: The experiences and destination planner data were defined as TypeScript constants within the page files. A production system would require a content management

system (CMS) or database-backed content API to enable non-technical staff to manage the catalogue without code deployments.

Payment integration: The system generated booking references and persisted reservation records but did not implement actual payment processing. Integration with Stripe or a similar payment gateway would be required for commercial operation, introducing PCI DSS compliance considerations.

Scalability: The file-system based JSON persistence used as a development fallback in the RMS would not survive horizontal scaling across multiple server instances. This was superseded by the Supabase integration in the primary path but remained as a development artefact.

AI prompt consistency: LLM responses exhibited natural variability that occasionally required post-processing to fit the expected display format. A more robust implementation would enforce structured JSON output through the Anthropic API's schema-constrained output feature.

Accessibility: Formal WCAG 2.1 AA compliance testing was not conducted. Elements such as the video hero background would require `prefers-reduced-motion` media query handling, and the dark colour palette would require contrast ratio verification for all text/background combinations.

Chapter 8: End-Project Report (Post-Mortem)

8.1 Project Management Reflection

The project was executed as a solo development effort over the course of the academic year. An informal agile approach was adopted, with feature development organised into two-week sprints with defined deliverables. The primary project management challenge was scope management: the luxury travel domain offered a practically unlimited surface area for feature development, and discipline in adhering to the MoSCoW prioritisation defined in Chapter 3 was essential to delivering a functional system within the available time.

The most significant schedule deviation occurred during the implementation of the booking system resilience mechanisms. What was initially estimated as a one-day task to wire the frontend booking form to the RMS expanded to approximately three days when the self-referential HTTP call issue was encountered and debugged. This experience reinforced the importance of end-to-end integration testing early in the development cycle, rather than unit-testing components in isolation and deferring integration until the final sprint.

8.2 Technical Lessons Learned

Environment configuration is a critical dependency: The discovery that the frontend proxy was calling its own port rather than the RMS highlighted the importance of explicit environment variable validation at startup. A production system should fail fast with a clear error message if `RMS_URL` is not configured, rather than silently routing to an incorrect endpoint.

Graceful degradation should be designed in, not added later: The initial implementation assumed the RMS would always be available. Adding the layered fallback mechanisms retrospectively required changes across three files. Designing for partial failure from the outset would have produced a cleaner implementation.

LLM APIs require billing management at the application level: The Anthropic API's credit depletion produced a connection error that was initially surfaced to the user as a confusing technical message. A production system should intercept API billing errors specifically and present them as a user-friendly service interruption notice, distinct from connectivity failures.

TypeScript strictness pays compound interest: The investment in strict TypeScript configuration prevented numerous potential runtime errors, particularly around nullable state in the Destination Planner and response typing in the booking flow. Several bugs that would have manifested as runtime `TypeError: Cannot read properties of undefined` were caught at compile time.

8.3 Personal Development Reflection

This project represented the developer's first experience building a full-stack application from conception to deployment-ready implementation. Prior experience was primarily limited to isolated frontend or backend exercises. The challenge of managing the

interaction between two Next.js applications — including the subtleties of server-to-server communication, environment variable scoping, and port allocation — provided a significantly deeper understanding of distributed web system architecture than any previous academic exercise.

The integration of the Anthropic Claude API was particularly instructive. Prompt engineering emerged as a distinct discipline requiring iterative refinement: early prompts produced unstructured narrative responses that did not parse cleanly into the UI components designed to receive them. The process of converging on a system prompt that reliably elicited structured, consistently formatted output developed a nuanced understanding of LLM behaviour that is directly applicable to future AI-integrated application development.

The aesthetic dimension of the project — designing a luxury brand identity and translating it into a consistent visual system — was unexpected preparation for commercial web development, where client expectations around visual quality frequently exceed the functional requirements as a source of project risk.

Chapter 9: Conclusions

This project delivered a full-stack luxury travel planning platform that met all stated functional requirements and demonstrated commercially viable design quality. The Samsara system comprised three distinct user-facing modules — an Experiences catalogue with multi-dimensional filtering and booking flow, an AI-powered Feelings Engine, and an interactive Destination Planner — backed by a separate Reservation Management System handling persistence and transactional email.

The primary technical contributions of the project were: (1) the design and implementation of a robust layered fallback architecture for the booking flow, ensuring user experience quality independent of backend availability; (2) the application of LLM-based AI to the luxury travel domain, demonstrating the feasibility of emotional-language travel recommendation as an alternative to conventional attribute-based search; and (3) the delivery of a consistent luxury design system implemented without third-party component libraries, demonstrating the capability to match commercial-grade design standards through first-principles CSS engineering.

Future development priorities would include: real payment processing integration (Stripe), a content management system for non-technical catalogue management, WCAG 2.1 AA accessibility audit and remediation, and performance optimisation through Next.js Image component adoption and video streaming optimisation.

The project successfully demonstrated the integration of modern full-stack JavaScript technologies, third-party API services, and thoughtful UX design in the service of a coherent, commercially-oriented product vision.

References

- Buhalis, D. and Law, R. (2008) 'Progress in information technology and tourism management: 20 years on and 10 years after the Internet – The state of eTourism research', *Tourism Management*, 29(4), pp. 609–623.
- Burke, R. (2002) 'Hybrid recommender systems: Survey and experiments', *User Modeling and User-Adapted Interaction*, 12(4), pp. 331–370.
- Dolnicar, S., Grabler, K., Grün, B. and Kulnig, A. (2011) 'Key drivers of airline loyalty', *Tourism Management*, 32(5), pp. 1020–1026.
- Fedosejev, A. (2015) *React.js Essentials*. Birmingham: Packt Publishing.
- Grand View Research (2024) *Luxury Travel Market Size, Share & Trends Analysis Report*. [Online] Available at: <https://www.grandviewresearch.com/industry-analysis/luxury-travel-market> (Accessed: January 2026).
- Howard, M. and LeBlanc, D. (2002) *Writing Secure Code*. 2nd edn. Redmond: Microsoft Press.
- Linden, G., Smith, B. and York, J. (2003) 'Amazon.com recommendations: Item-to-item collaborative filtering', *IEEE Internet Computing*, 7(1), pp. 76–80.
- Monus, A. (2022) *Next.js Quick Start Guide*. Birmingham: Packt Publishing.
- Newman, S. (2015) *Building Microservices*. Sebastopol: O'Reilly Media.
- Pine, B.J. and Gilmore, J.H. (1999) *The Experience Economy: Work Is Theatre & Every Business a Stage*. Boston: Harvard Business School Press.
- Prahalad, C.K. and Ramaswamy, V. (2004) 'Co-creating unique value with customers', *Strategy & Leadership*, 32(3), pp. 4–9.
- Vargo, S.L. and Lusch, R.F. (2008) 'Service-dominant logic: Continuing the evolution', *Journal of the Academy of Marketing Science*, 36(1), pp. 1–10.
- Wang, L., Yang, N., Wei, F., Tang, B. and Zhao, J. (2023) 'Large language models are zero-shot reasoners', *Advances in Neural Information Processing Systems*, 36.

Appendices

Appendix A: Source Code Repository

The complete source code for the Samsara project is hosted on Plymouth University's OneDrive at the following location:

[Plymouth OneDrive Link — to be inserted by student prior to submission]

The repository contains two applications:

- `samsara/samsara-frontend/` — Consumer-facing Next.js application (port 3001)
- `samsara/samsara-rms/` — Reservation Management System (port 3000)

A `README.md` at the repository root documents the local development setup procedure, including environment variable configuration requirements.

Appendix B: Environment Variable Reference

`samsara-frontend/.env.local`

Variable	Purpose
<code>RMS_URL</code>	Base URL of the <code>samsara-rms</code> application (default: <code>http://localhost:3000</code>)
<code>NEXT_PUBLIC_ANTHROPIC_API_KEY</code>	Anthropic API key for Feelings Engine

`samsara-rms/.env.local`

Variable	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase project URL
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Supabase service-role key (server-side only)
<code>RESEND_API_KEY</code>	Resend API key for transactional email
<code>EMAIL_FROM</code>	Sender address for confirmation emails

Appendix C: System Startup Procedure

```
# Install all dependencies
npm install # root
cd samsara-rms && npm install
cd ../samsara-frontend && npm install

# Start both applications concurrently from root
npm run dev # starts samsara-rms on :3000, samsara-frontend on :3001
```

```
# Or individually:  
npm run dev:rms      # samsara-rms on port 3000  
npm run dev:frontend # samsara-frontend on port 3001
```

Appendix D: Booking Reference Format

Booking references are generated in the format: `ITW-[BASE36_TIMESTAMP]-[RANDOM_3CHARS]`

Example: `ITW-M5X2A-K7F`

The ITW prefix stands for *Into The Wild* (Samsara's sub-brand for experiential bookings). Timestamp encoding in base-36 provides compact, alphanumeric references that are visually distinct from purely numeric identifiers and easy for customer service staff to communicate verbally.